Data Mining for Smarter Airbnb Price Prediction

Mohith Sadashiva Reddy
Computer Science
Arizona State University
Tempe, USA
msadash1@asu.edu

Ganesh Kumar Chintapalli
Data Science
Arizona State University
Tempe, USA
gchintap@asu.edu

Abdul Qayyum Abdul Rahim Khan
Data Science
Arizona State University
Tempe, USA
akhan228@asu.edu

***Abstract*—Setting a fair nightly price on Airbnb is difficult because price depends on many factors such as location, room type, reviews, and demand patterns. In this project, we use the Airbnb 2023 USA dataset containing about 232,000 listings across 27 cities to build a price prediction system. We clean the data, handle missing values, reduce extreme outliers, and create new features such as geographical clusters from latitude and longitude, review recency, and text-based signals from listing titles. We then train and compare several regression models including Linear Regression, Random Forest, Gradient Boosting, and XGBoost. XGBoost performs best and, after tuning and feature improvements, achieves an R^2 of about 0.5156 with mean absolute error (MAE) around $58.30 and root mean squared error (RMSE) around $89.49. These results show that data mining methods can help hosts choose realistic prices and improve booking decisions for both hosts and guests.**

I. Introduction

Online short-term rental platforms such as Airbnb have changed the way people book accommodation. Unlike traditional hotels, each Airbnb listing is unique. Price depends not only on straightforward attributes such as the number of beds or the type of room but also on location, guest reviews, host reputation and many subtle listing details. As a result, understanding and predicting Airbnb prices is a challenging data mining problem.

In practice many hosts still set their nightly prices by guesswork or by loosely copying competitors. If the price is set too high, the listing attracts fewer bookings and stays empty. If the price is set too low, the host loses potential revenue and may attract bookings that are not sustainable in the long term. From the guest perspective, inconsistent pricing makes it difficult to judge whether a listing is fairly priced compared with similar options nearby.

Our aim in this project is to ask a simple question: *given the historical Airbnb data can we build a model that predicts a reasonable nightly price for a listing?* We treat this as a supervised regression task and build a complete data mining pipeline that starts from raw listings for many U.S. cities and ends with a tuned predictive model.

We use the Airbnb USA 2023 dataset, which contains approximately 232,000 listings from 27 major cities. The dataset is noisy and heterogeneous: it contains missing values, extreme luxury listings, and a wide mix of room types, host behaviours, and neighbourhoods. To deal with this, we design a pipeline that combines several ideas:

- systematic data cleaning and outlier handling
- feature engineering for location, review activity, host information and listing text
- a unified preprocessing pipeline that can handle both numeric and categorical variables
- a comparison of several regression models with a focus on gradient boosting.

Through experiments we find that tree-based ensemble models, like XGBoost, significantly outperform a simple linear baseline. After tuning and adding richer features, our final XGBoost model explains roughly half of the variation in nightly prices and has an average error of about $58 per listing. While this is not perfect, it shows that a fairly simple model with good features can provide useful guidance to both hosts and guests.

The main contributions of this paper are:

- An end-to-end data mining pipeline for Airbnb price prediction on a large, real-world U.S. dataset.
- A set of engineered features that successfully combine geospatial information, review behaviour, host signals, and text-based cues from listing titles.
- An empirical comparison of multiple regression models and an ablation study showing how feature engineering and tuning improve performance.

The remainder of this paper is organized as follows. Section II defines the dataset, features, and formal problem. Section III presents an overview of our proposed system. Section IV explains the technical details of preprocessing, feature engineering, and modeling. Section V describes the experimental setup and evaluation. Section VI discusses results and insights, Section VII briefly reviews related work, and Section VIII concludes the paper.

II. Data, Definitions, and Problem Statement

A. Dataset Description

We use the Airbnb USA 2023 dataset, which contains approximately 232,000 active listings from 27 major cities in the United States. Each row represents a single listing and each column describes one property of that listing. Example columns include:

- `price`: advertised nightly price in U.S. dollars,
- `room_type`: entire home, private room, shared room, or hotel room,

TABLE I
OVERVIEW OF THE AIRBNB USA 2023 DATASET USED IN THIS STUDY.

Property	Value
Source	Airbnb USA 2023 listings
Number of listings	$\sim$ 232,000
Number of cities	27 major U.S. cities
Main geo fields	Latitude, longitude, city
Target variable	Nightly price (USD)
Main attributes	Room type, min nights, reviews, availability
Time field	Date of last review

- `minimum_nights`: minimum consecutive nights the guest must book,
- `number_of_reviews`: total number of reviews left by guests,
- `reviews_per_month`: monthly review rate,
- `calculated_host_listings_count`: number of listings managed by the same host,
- `availability_365`: number of days in the year when the listing is available,
- `last_review`: date of the most recent review,
- `latitude`, `longitude`: geographical coordinates of the listing,
- `name`: short text title of the listing.

The raw data also contains several other fields that are either duplicates, extremely sparse, or less relevant for our goals. During preprocessing we discard such columns and focus on a subset that is informative and manageable.

B. Target Variable

The prediction target in our project is the nightly price of a listing. In the raw dataset, prices are heavily skewed: most listings are moderately priced, but a small number are extremely expensive luxury properties. Training directly on such a skewed target can cause models to focus too much on rare extremes.

To stabilise training, we transform the price using the natural logarithm:

$$y = \log(\text{price} + 1).$$

The $+1$ avoids taking the log of zero. Models are trained to predict y; at evaluation time we map predictions back to the original scale using $\widehat{\text{price}} = \exp(\hat{y}) - 1$. All reported evaluation metrics are computed on the dollar scale.

C. Feature Groups

We group our features into several categories to make the system easier to design and interpret:

- **Listing attributes:** room type, minimum nights, availability_365, and the number of accommodates (if available).
- **Host attributes:** host identifier (encoded) and the number of listings managed by that host.
- **Review signals:** number of reviews, reviews per month, and the time since the last review.
- **Geospatial features:** city and a cluster label derived from latitude and longitude.

TABLE II
MAIN FEATURE GROUPS USED IN THE FINAL MODELING PIPELINE.

Group	Example Features and Description
Listing	Room type, minimum nights, availability_365
Host	Host ID (target-encoded), host listing count
Reviews	Number of reviews, reviews per month, recent reviews
Geo	City, KMeans-based geo_cluster from lat/long
Time	Days since last review
Text	TF–IDF features from listing title (`name`)
Amenity	Flags like has_pool, has_view, is_luxury, has_kitchen

- **Time-based features:** days since the last review, which describes how recently guests have interacted with the listing.
- **Text and amenity features:** TF–IDF features from the listing title and simple binary flags that indicate the presence of keywords such as “pool”, “view”, or “luxury”.

D. Formal Problem Statement

Let X be the feature vector describing a listing and let y be the transformed price defined above. From a data mining perspective, we are given a dataset

$$\mathcal{D} = \{(X_i, y_i)\}_{i=1}^{N}$$

and we aim to learn a regression function $f : X \to \mathbb{R}$ such that $\hat{y}_i = f(X_i)$ is as close as possible to y_i.

The main challenges are:

- **Heterogeneity:** listings come from many cities, neighbourhoods, and host types, creating varied patterns.
- **Skew and outliers:** a small number of extreme luxury listings dominate the highest price range.
- **High-cardinality categories:** variables such as host ID and neighbourhood have thousands of distinct values.
- **Noisy text fields:** listing titles contain useful information but are short and unstructured.

Our goal is to design features and models that deal with these issues while staying relatively simple and interpretable.

III. OVERVIEW OF THE PROPOSED SYSTEM

Our system follows a standard data mining pipeline with stages for data cleaning, feature engineering, preprocessing, model training, and evaluation. A high-level overview is shown in Fig.1.

Figure1.png

Fig. 1. Overview of the proposed Airbnb price prediction pipeline.

A. Pipeline Stages

The pipeline consists of the following main steps:

1) **Data loading and inspection:** We load the Airbnb USA 2023 dataset and perform initial exploration to understand missing values, price ranges, and basic statistics.
2) **Data cleaning:** We remove clearly invalid records (such as listings with zero price) and address missing values in review-related fields.
3) **Feature engineering:** We create new features that capture geospatial structure, review recency, and textual cues from listing titles, along with simple amenity flags.
4) **Preprocessing and encoding:** We build a unified preprocessing pipeline that encodes categorical variables, applies TF–IDF to text, and passes through or scales numeric features.
5) **Model training and selection:** We train several regression models on the processed features and compare their performance. XGBoost is selected as the champion model.
6) **Model tuning and ablation:** We tune the XGBoost model and perform ablation studies to measure the impact of feature engineering and filtering.
7) **Evaluation and interpretation:** We evaluate the final model on a held-out test set and analyse feature importance and prediction behaviour.

This modular design makes it easy to run experiments, add or remove features, and swap models without rewriting large parts of code.

B. Logical Architecture

From a logical perspective, the system can be seen as three layers:

- a *data preparation layer* that transforms raw CSV files into a clean feature matrix,
- a *modeling layer* that takes this matrix and fits regression models,
- an *evaluation layer* that computes metrics, plots, and case studies.

In a practical deployment scenario, the data preparation and modeling layers would be implemented as reusable components. A host-facing interface could collect listing details, pass them through the preprocessing pipeline, and query the trained model for a suggested price.

IV. Technical Details of the Proposed Approach

A. Data Preprocessing

Real-world datasets are rarely clean. The Airbnb dataset contains missing values, inconsistent entries, and extreme outliers. We perform several preprocessing steps before modeling.

1) Handling Invalid and Missing Values: We first remove listings with price equal to zero, since such records do not represent valid nightly prices. For review-related columns such as `reviews_per_month`, missing values typically indicate listings with no reviews. We therefore replace missing values with 0 in these fields.

Some listings also have missing or invalid dates in the `last_review` column. For these, we treat the review recency as unknown and later encode it as a separate category or a large value of days since last review.

2) Outlier Reduction: The price distribution is highly skewed, with a small number of extremely expensive listings. In early experiments these extreme values caused models to allocate too much capacity to rare cases. To reduce this effect, we apply two strategies:

- we cap the price at the 99th percentile when computing summary statistics and visualisations, and
- in some experiments we drop the top 5% most expensive listings and listings with very large minimum night requirements (e.g., ≥ 10 nights), since these often behave more like long-term rentals than typical short stays.

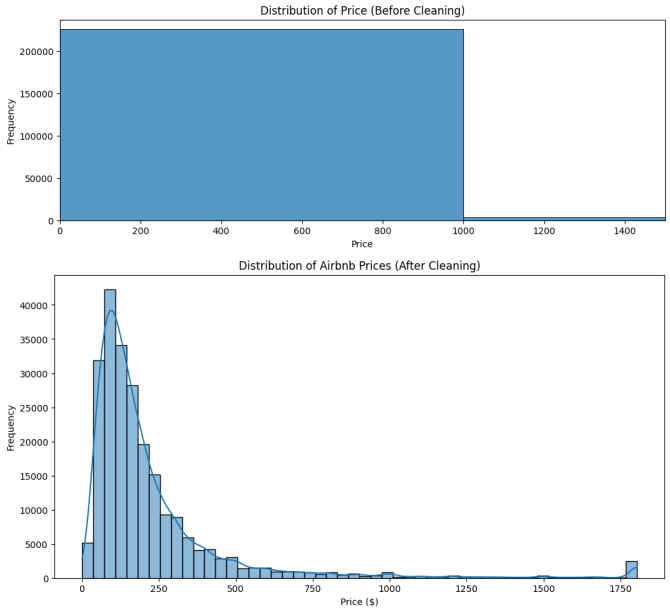

Fig. 2. Distribution of nightly prices before (top) and after (bottom) log transformation.

Fig. 2 illustrates the price distribution before and after log transformation. The transformed distribution is much closer to normal and is therefore easier for models to handle.

B. Geospatial Feature Engineering

Location is one of the most important factors affecting price. However, raw latitude and longitude values are not easy for tree-based models to interpret directly. They capture distance but not clearly defined neighbourhoods.

To encode neighbourhood-like information, we apply KMeans clustering within each city:

1) For each city, we extract the latitude and longitude of its listings.
2) We select a number of clusters (for example between 8 and 20 depending on the number of listings).
3) We run KMeans to partition the city into compact regions.

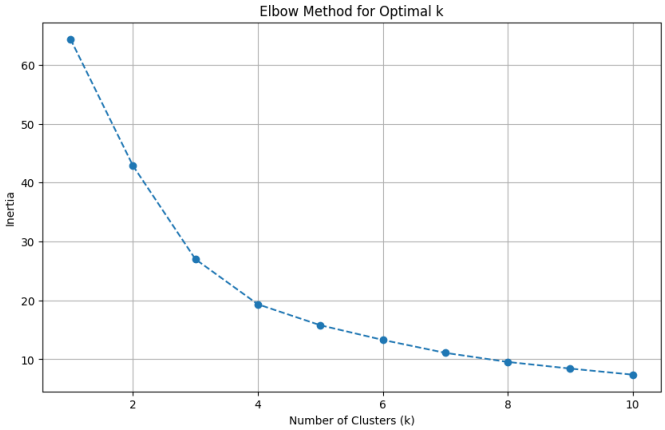

Fig. 3. Finding optimal k value

Each listing is then assigned a cluster label. We combine the city name and cluster index to create a categorical feature `geo_cluster`. This feature acts as a simple neighbourhood identifier and allows the model to capture local price patterns.

We also visualise the clusters on a map to verify that they look reasonable (e.g., coastal clusters for beach areas, central clusters for downtown). An example map can be included as Fig.4.

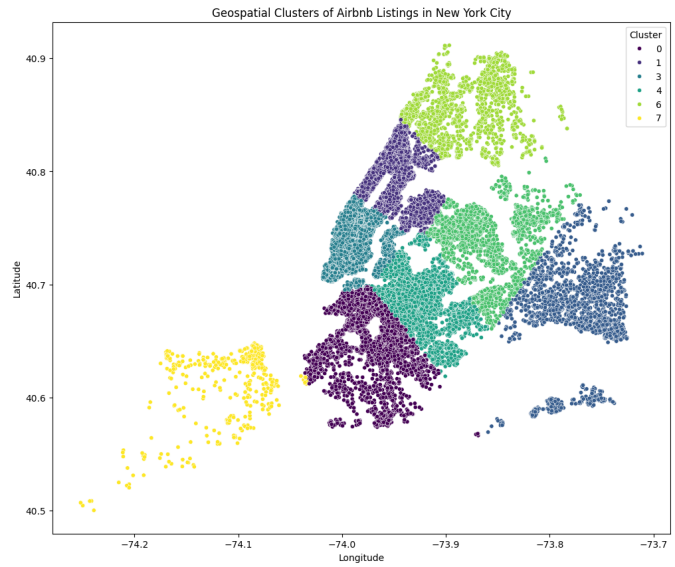

Fig. 4. Example KMeans-based geospatial clusters for Airbnb listings in a sample city.

C. Text and Amenity Features

The listing title (`name`) often contains useful information, such as:

> "Cozy studio near downtown", "Luxury oceanfront condo with pool", or "Budget private room".

We use a TF–IDF vectorizer to convert the title into numeric features. We include unigrams and bigrams and limit the vocabulary to the most frequent terms (for example 300 features) to keep the model efficient.

In addition to TF–IDF we construct simple binary amenity flags based on keyword matching. For example, if the title contains words like "pool" or "swimming", we set `has_pool` to 1. Similarly we set flags for `has_view`, `is_luxury`, and `has_kitchen`. These features make it easier for the model to learn consistent price differences between basic and premium listings.

D. Encoding Categorical Variables

The dataset contains both low-cardinality and high-cardinality categorical variables.

Variables such as `room_type` and `city` have a small number of possible values and are encoded with standard one-hot encoding. This creates one binary feature per category.

Variables such as `host_id` and neighbourhood names can take thousands of values. For these we apply target encoding:

each category is replaced by the mean of the target variable (the log price) for that category in the training data. We use smoothing techniques to avoid overfitting small categories.

E. Preprocessing Pipeline

To keep the implementation organised, we use a `ColumnTransformer` to combine all preprocessing steps into a single pipeline:

- the text column is passed through a TF–IDF vectorizer,
- high-cardinality categorical columns are passed through a target encoder,
- low-cardinality categorical columns are passed through a one-hot encoder,
- numeric columns (including engineered features) are passed through unchanged or standardised if needed.

This pipeline is then wrapped around the regression model using a scikit-learn `Pipeline` object. This design ensures that any cross-validation or grid search correctly applies preprocessing inside each fold, preventing data leakage.

F. Regression Models

We consider the following regression models:

- **Linear Regression:** a simple baseline that models the log price as a weighted sum of features.
- **Random Forest Regressor:** an ensemble of decision trees trained on bootstrap samples with random feature selection at each split.
- **Gradient Boosting Regressor:** an ensemble that builds trees sequentially, each correcting the errors of the previous ones.
- **XGBoost Regressor:** an improved gradient boosting algorithm with regularization and efficient handling of sparse features.

Early experiments show that Linear Regression performs poorly, which suggests that the relationship between features and price is highly non-linear. Random Forest and Gradient Boosting perform better but are slower to train on the full dataset. XGBoost gives the best trade-off between training time and accuracy and is therefore chosen as the main model to tune.

V. Experiments

A. Train–Test Split

We split the cleaned dataset into a training set (80%) and a test set (20%) using a fixed random seed to make results reproducible. All preprocessing steps described in Section IV are fitted on the training data only and then applied to the test data through the same pipeline.

B. Hyperparameter Tuning

For XGBoost we perform a grid search over a few important hyperparameters:

- the maximum depth of each tree,
- the number of trees (estimators),
- the learning rate,
- the subsample ratio for rows.

We tune these parameters using cross-validation on the training set while monitoring the mean squared error on the validation folds. The final configuration uses a learning rate of 0.1, maximum depth of 10, around 400 trees, and a subsample ratio of 0.8.

C. Evaluation Metrics

To evaluate model performance we use:

- the coefficient of determination R^2, which measures the proportion of variance in price explained by the model,
- mean absolute error (MAE), which measures the average absolute difference between predicted and actual prices,
- root mean squared error (RMSE), which penalises larger errors more strongly.

Because we train models on log-transformed prices, we always convert predictions back to the original dollar scale before computing these metrics.

D. Baseline Model Comparison

We first compare the four main regression models without aggressive filtering or advanced text features. Table III summarises their approximate performance on the test set.

TABLE III
Comparison of main regression models on the test set.

Model	R^2	MAE ($)	RMSE ($)
Linear Regression	0.1495	135.70	244.09
Random Forest	0.2180	109.73	234.0500
Gradient Boosting	0.2038	108.9700	236.1600
XGBoost (baseline)	0.2628	104.8300	227.2500

The Linear Regression baseline explains very little of the variance and produces high errors. Random Forest and Gradient Boosting roughly double the R^2 but still leave a lot of variation unexplained. XGBoost slightly improves on these while training more quickly, so we focus on it for further refinement.

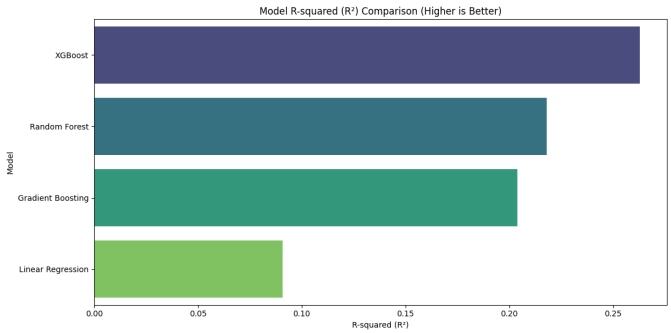

Fig. 5. Comparison of R^2 scores for the main regression models.

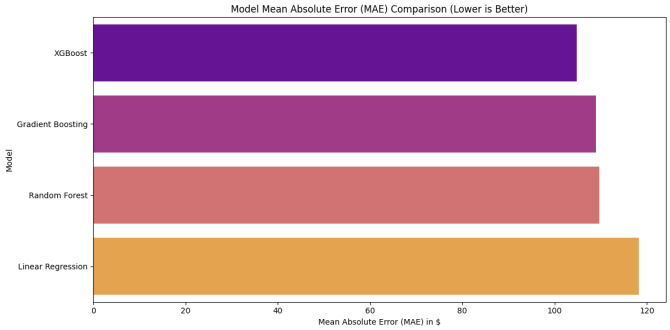

Fig. 6. Comparision of MAE for the main regression models

E. Ablation Study: Impact of Features and Filtering

Next we analyse how much our feature engineering and filtering steps help XGBoost. Table IV shows an ablation study comparing a tuned XGBoost model without text/amenity features against our final model with all features and stricter filtering.

TABLE IV
ABLATION STUDY FOR TUNED XGBOOST WITH AND WITHOUT TEXT/AMENITY FEATURES AND FILTERING.

Configuration	R^2	**MAE ($)**	**RMSE ($)**
Tuned XGBoost (no text)	0.3604	97.59	211.67
Final XGBoost (with text)	0.5156	58.30	89.49

We observe that adding text and amenity features, along with better handling of extreme listings, increases R^2 by about 0.1552 and reduces MAE and RMSE by roughly \$39 and \$122 respectively. This confirms that feature engineering contributes significantly to model quality.

F. Case Studies

To better understand model behaviour, we inspect a few individual predictions. For example, a mid-range "entire home" in a central neighbourhood with many recent reviews tends to be predicted accurately: the model captures the effect of room type, active reviews, and high-demand location.

In contrast, very unusual listings such as large luxury villas or unique properties sometimes show larger prediction errors. These listings are rare and often have features that are not captured directly in the data, such as special design, brand reputation, or personal services. These case studies highlight both the strengths and the limitations of the approach.

VI. RESULTS AND DISCUSSION

A. Overall Performance

Our final tuned XGBoost model achieves:

$$R^2 = 0.5156, \quad \text{MAE} \approx \$58.30, \quad \text{RMSE} \approx \$89.49.$$

An R^2 of roughly 0.52 means that the model can explain about half of the variation in nightly prices across the dataset. Considering the heterogeneity of listings and the simplicity of the features, this is a reasonable level of performance. The MAE of about \$58 indicates that on average the model's predictions are within \$60 of the true price. For many hosts, this is useful guidance: it can suggest a competitive price band even if it is not exact.

B. Feature Importance

We extract feature importance scores from the final XGBoost model. Fig.7 shows the top contributing features. Location-related features such as city and `geo_cluster` are among the most important, confirming that neighbourhood and local demand strongly influence price. Listing attributes such as room type and availability also have high importance.

Text and amenity features contribute meaningfully. For example, keywords like "view", "beach", "downtown", or "luxury" are associated with higher prices, while terms like "budget" and "shared" tend to correlate with lower prices. This shows that even simple text processing can extract useful signals from short listing titles.

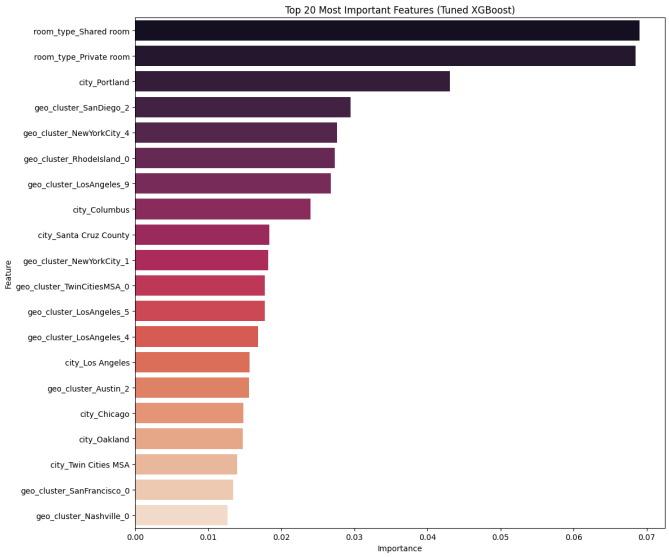

Fig. 7. Top feature importances in the final XGBoost model.

C. Error Analysis

We analyse prediction errors by grouping listings into different categories.

1) By Price Level: For lower and medium price ranges (for example \$50–\$200 per night), errors are moderate and relatively stable. For very high prices (above roughly \$500 per night), errors increase. This is expected because there are fewer training examples of luxury properties and they are more diverse.

2) By Room Type: The model performs slightly better for entire homes and private rooms than for shared rooms or hotel rooms. Shared rooms are relatively rare and have diverse conditions, while hotel-like listings may have different pricing strategies that are not fully captured by the dataset.

3) By City: Performance also varies by city. In cities with many listings and relatively consistent neighbourhood patterns, the model performs better. In smaller markets or

cities with very mixed inventory, predictions are noisier. This suggests that city-specific models or adding external city-level information (such as average income or tourism statistics) could further help.

D. Practical Interpretation

From a practical perspective, our model does not replace host judgement, but it can act as a suggestion engine. A host could provide listing details and receive a recommended price range based on similar listings in their city and neighbourhood. They can then adjust this suggestion based on their own constraints, such as owner preferences or cleaning costs.

The model is also useful for analysing the market. Feature importance and partial dependence plots could show, for example, how much extra price is associated with being in a certain cluster or having particular amenities. This information can help hosts decide which improvements to make to their property.

VII. Related Work

Price prediction has been widely studied in domains such as real estate, hotel bookings, and ride sharing. Classical approaches use linear regression or hedonic pricing models with features such as location, size, and quality indicators. More recent work employs tree-based ensembles and gradient boosting, which handle non-linear interactions and mixed data types more effectively.

For Airbnb specifically, several studies show that location, property type, and review scores are consistent predictors of price. Some works enrich the feature space with neighbourhood-level statistics or geographical distance to city centres and tourist attractions. Others incorporate textual information from listing descriptions using TF–IDF or word embeddings.

Our work follows this line of research but focuses on building a practical, interpretable pipeline for a large U.S. dataset. Instead of using complex deep learning models, we show that careful feature engineering combined with gradient boosting (XGBoost) can already achieve strong performance while remaining relatively easy to train and deploy.

VIII. Conclusion and Future Scope

In this paper we presented a data mining approach to predicting nightly prices for Airbnb listings in the United States. Using the Airbnb USA 2023 dataset, we constructed an end-to-end pipeline that includes data cleaning, feature engineering, preprocessing, model training, and evaluation.

We showed that simple linear models are not sufficient for this task, while non-linear ensemble methods perform significantly better. Among the models we tested, XGBoost offered the best trade-off between accuracy and efficiency. After tuning and adding geospatial, review, text, and amenity features, our final model achieved an R^2 of 0.5156, MAE of \$58.30, and RMSE of \$89.49 on a held-out test set.

The results demonstrate that data mining and machine learning can provide useful support for hosts who need guidance on pricing and for platforms that want to surface realistic price suggestions. At the same time, the remaining error and variability highlight that pricing is influenced by factors beyond the available data, including host strategies and local events.

There are several promising directions for future work:

- **Richer models for categorical data:** exploring algorithms such as CatBoost, which handle categorical variables natively and may improve both accuracy and interpretability.
- **External geospatial information:** incorporating distance to city centres, beach fronts, public transport, or points of interest to refine the notion of location.
- **Temporal dynamics:** modelling seasonal patterns, holidays, and major events to capture demand fluctuations over time.
- **City-specific or hierarchical models:** building models tailored to individual cities or using hierarchical approaches that share information across cities while allowing city-level differences.
- **User-facing tools:** wrapping the trained model into a simple web interface where hosts can input listing details and receive suggested price ranges along with explanations.

Overall, our study suggests that even relatively straightforward machine learning techniques, when combined with thoughtful feature engineering, can provide valuable insights in the Airbnb pricing domain.

IX. References